Management Platform

Documento Técnico — Plataforma de Gestión de Proyectos y Tareas

Marzo 2026 · Clean Architecture · .NET 8 · PostgreSQL · Docker

Tabla de Contenidos

- 1. Descripción General
- 2. Tecnologías Utilizadas
- 3. Estructura del Proyecto
- 4. Descripción de Módulos
- 5. Flujo del Sistema
- 6. Fragmentos de Código Relevantes
- 7. API — Endpoints
- 8. Despliegue con Docker
- 9. Pruebas Unitarias

---

1. Descripción General

Management Platform es una aplicación web full-stack para la gestión de proyectos y tareas, desarrollada como assessment técnico. Implementa **Clean Architecture** con separación estricta de responsabilidades en capas independientes, garantizando mantenibilidad, testabilidad y escalabilidad.

Características principales:

- Autenticación y autorización con JWT Bearer Tokens
- CRUD completo de Proyectos y Tareas
- Ciclo de vida controlado por reglas de dominio (Draft → Active → Completed)
- Paginación y filtrado de proyectos
- Interfaz web MVC + API REST documentada con Swagger
- Contenedores Docker para todos los servicios

---

2. Tecnologías Utilizadas

Categoría	Tecnología	Versión / Detalle
Framework Backend	ASP.NET Core	.NET 8 — API REST + MVC
ORM	Entity Framework Core	8.x — code-first migrations
Base de Datos	PostgreSQL	16 (vía Npgsql provider)
Autenticación	JWT Bearer	System.IdentityModel.Tokens.Jwt
Hashing de Contraseñas	BCrypt.Net-Next	salted bcrypt hashing
Documentación API	Swagger / Swashbuckle	OpenAPI 3.0
Contenedores	Docker + Docker Compose	Multi-service orchestration
Testing	xUnit + Moq	Pruebas unitarias de dominio
Lenguaje	C#	C# 12 / .NET 8

3. Estructura del Proyecto

```
Management_Platform.sln
├── Platform.Domain          // Núcleo: entidades, enums, interfaces, excepcione
│   ├── Entities/
│   │   ├── Project.cs
│   │   ├── TaskItem.cs
│   │   └── User.cs
│   ├── Enums/               // ProjectStatus, Priority
│   ├── Exceptions/          // DomainException
│   └── Interfaces/          // IProjectRepository, ITaskRepository, IUnitOfWork
│
├── Platform.Application     // Casos de uso y lógica de aplicación
│   ├── DTOs/                // CreateProjectDto, ProjectDto, TaskDto...
│   ├── Interfaces/          // IProjectService, ITaskService, IAuthService
│   ├── Services/            // ProjectService, TaskService
│   ├── Common/Models/       // PagedResult<T>
│   └── DependencyInjection.cs
│
├── Platform.Infrastructure  // Persistencia y servicios externos
│   ├── Data/
│   │   └── ApplicationDbContext.cs
│   ├── Repositories/        // ProjectRepository, TaskRepository, UserRepositor
│   ├── Services/            // AuthService (JWT + BCrypt)
│   ├── Migrations/
│   ├── UnitOfWork.cs
│   └── DependencyInjection.cs
│
├── Platform.Api             // REST API — punto de entrada HTTP
│   ├── Controllers/         // ProjectsController, TasksController, AuthControl
│   ├── Program.cs           // Configuración DI, JWT, Swagger, EF
│   ├── Dockerfile
│   └── appsettings.json
│
├── Platform.Web             // Interfaz web MVC con Razor Views
│   ├── Controllers/
│   ├── Views/
│   ├── Models/
│   ├── Dockerfile
│   └── Program.cs
│
├── Platform.Tests           // Pruebas unitarias (xUnit)
│   └── Domain/
│       └── ProjectTests.cs
```

```
  │
  └── docker-compose.yml       // Orquestación: db + api + web
```

4. Descripción de Módulos

4.1 Platform.Domain

Capa más interna. No depende de ninguna otra capa. Contiene las reglas de negocio puras.

Platform.Domain ← sin dependencias externas

- `Project.cs` — Entidad principal con ciclo de vida (Draft/Active/Completed) y reglas de negocio encapsuladas
- `TaskItem.cs` — Tarea con prioridad, orden único por proyecto, y estado de completitud
- `User.cs` — Usuario con email único y contraseña hasheada
- `DomainException` — Excepción de dominio lanzada ante violaciones de reglas de negocio
- `IProjectRepository / ITaskRepository / IUnitOfWork` — Contratos que la infraestructura implementa

4.2 Platform.Application

Orquesta los casos de uso. Depende únicamente del dominio. No conoce EF Core ni la base de datos.

- `ProjectService` — Crear, editar, eliminar, paginar, activar y completar proyectos
- `TaskService` — CRUD de tareas, cambio de estado y reorden
- **DTOs** — Objetos que cruzan la frontera entre capas (sin lógica de negocio)
- `PagedResult<T>` — Modelo genérico de respuesta paginada

4.3 Platform.Infrastructure

Implementa los contratos del dominio. Contiene todo el código de acceso a datos y servicios externos.

- `ApplicationDbContext` — Configura mapeo ORM, restricciones e índices únicos
- `ProjectRepository / TaskRepository` — Consultas EF Core, filtrado, paginación con `Skip/Take`
- `UnitOfWork` — Agrupa repositorios y centraliza `SaveChangesAsync()`
- `AuthService` — Registro/login con BCrypt + generación de JWT

4.4 Platform.Api

Capa de presentación REST. Recibe HTTP, delega a Application, devuelve JSON.

- `ProjectsController` — 7 endpoints: CRUD + activate + complete + search
- `TasksController` — 5 endpoints: CRUD + complete + reorder
- `AuthController` — register + login
- `Program.cs` — Configuración de DI, JWT, Swagger, middleware

4.5 Platform.Web

Interfaz de usuario con ASP.NET Core MVC + Razor Views. Consume la API internamente.

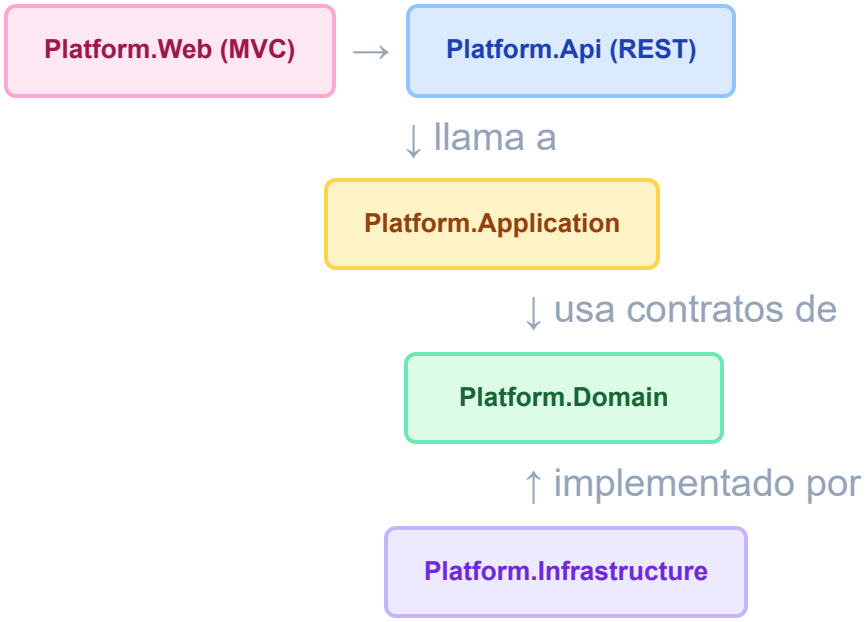
4.6 Platform.Tests

Pruebas unitarias con xUnit que verifican las reglas de negocio del dominio en aislamiento.

---

5. Flujo del Sistema

5.1 Arquitectura en Capas



5.2 Flujo de una Petición HTTP (ejemplo: Crear Proyecto)

1. Cliente HTTP → 2. JWT Middleware → 3. ProjectsController → 4. IProjectService → 5. Project.Create() [Dominio] → 6. IUnitOfWork / Repository → 7. EF Core → PostgreSQL → 8. HTTP 201 Created

5.3 Flujo de Autenticación

POST /api/auth/login → Buscar Usuario en DB → BCrypt.Verify() → Generar JWT (claims) → Retornar Token

5.4 Ciclo de Vida de un Proyecto

Draft → (requiere ≥1 tarea) Active → (todas las tareas completadas) Completed

Un proyecto Completed no puede reactivarse. Estas reglas viven en la entidad de dominio, no en el controlador.

---

6. Fragmentos de Código Relevantes

6.1 Entidad de Dominio: Project.cs

Demuestra encapsulamiento total, factory method, y reglas de negocio dentro de la entidad.

```
public class Project
{
    public Guid Id { get; private set; }
    public string Name { get; private set; } = string.Empty;
    public ProjectStatus Status { get; private set; }
    public ICollection<TaskItem> Tasks { get; private set; } = new List<TaskItem

    // Constructor privado: solo se puede crear mediante el factory method.
    private Project() { }

    // Factory method: valida y construye la entidad en estado válido.
    public static Project Create(string name, string description, Guid userId)
    {
        if (string.IsNullOrWhiteSpace(name))
            throw new DomainException("El nombre del proyecto es obligatorio.");

        return new Project {
            Id = Guid.NewGuid(), UserId = userId,
            Name = name.Trim(), Description = description?.Trim() ?? string.Empt
            Status = ProjectStatus.Draft, CreatedAt = DateTime.UtcNow
        };
    }

    // Regla de negocio: un proyecto necesita al menos 1 tarea para activarse.
    public void Activate(int taskCount)
    {
        if (taskCount == 0)
            throw new DomainException("El proyecto debe tener al menos una tarea
        if (Status == ProjectStatus.Completed)
            throw new DomainException("Un proyecto completado no puede reactivar
        Status = ProjectStatus.Active;
    }

    // Regla de negocio: todas las tareas deben completarse antes de cerrar.
    public void Complete(int totalTasks, int completedTasks)
    {
        if (Status != ProjectStatus.Active)
            throw new DomainException("Solo un proyecto activo puede completarse
        if (totalTasks == 0 || completedTasks < totalTasks)
            throw new DomainException("Todas las tareas deben completarse para c
        Status = ProjectStatus.Completed;
    }
}
```

6.2 Servicio de Aplicación: ProjectService.cs

Orquesta los casos de uso usando el `IUnitOfWork`. No sabe nada de EF Core ni HTTP.

```
public class ProjectService : IProjectService
{
    private readonly IUnitOfWork _uow;
    public ProjectService(IUnitOfWork uow) => _uow = uow;

    // Activa el proyecto: delega la regla de negocio al dominio.
    public async Task ActivateAsync(Guid id)
    {
        var project = await _uow.Projects.GetByIdAsync(id);
        if (project == null) throw new DomainException("Proyecto no encontrado."

        var tasks = await _uow.Tasks.GetByProjectIdAsync(id);
        project.Activate(tasks.Count()); // Regla de negocio en entidad

        await _uow.Projects.UpdateAsync(project);
        await _uow.SaveChangesAsync();
    }

    // Paginación con filtro de estado opcional.
    public async Task<PagedResult<ProjectDto>> GetPagedAsync(int page, int pageS
    {
        var (items, totalCount) = await _uow.Projects.GetPagedAsync(page, pageSi
        return new PagedResult<ProjectDto>(items.Select(MapToDto), totalCount, p
    }
}
```

6.3 Repositorio: ProjectRepository.cs

Implementa las consultas EF Core. La capa de aplicación solo conoce la interfaz, nunca este código.

```
public async Task<(IEnumerable<Project> Items, int TotalCount)>
    GetPagedAsync(int page, int pageSize, string? statusFilter)
{
    var query = _context.Projects.AsQueryable();

    // Filtro dinámico por estado si se provee.
    if (!string.IsNullOrWhiteSpace(statusFilter) &&
        Enum.TryParse<ProjectStatus>(statusFilter, true, out var status))
    {
        query = query.Where(p => p.Status == status);
    }

    // Contar total antes de paginar para calcular metadatos de paginación.
    var totalCount = await query.CountAsync();
    var items = await query
        .OrderBy(p => p.Name)
        .Skip((page - 1) * pageSize).Take(pageSize)
        .Include(p => p.Tasks)   // Carga eager de tareas relacionadas
        .ToListAsync();

    return (items, totalCount);
}
```

6.4 Controlador REST: ProjectsController.cs

Punto de entrada HTTP. Extrae el `userId` del JWT, delega al servicio, maneja errores de dominio.

```
[ApiController][Route("api/projects")][Authorize]
public class ProjectsController : ControllerBase
{
    private readonly IProjectService _projectService;
    public ProjectsController(IProjectService projectService) => _projectService

    // POST /api/projects — Extrae userId del JWT y crea el proyecto.
    [HttpPost]
    public async Task<IActionResult> Create([FromBody] CreateProjectDto dto)
    {
        try {
            var userId = Guid.Parse(User.FindFirstValue(ClaimTypes.NameIdentifie
            var project = await _projectService.CreateAsync(dto with { UserId =
            return CreatedAtAction(nameof(GetTasks), new { id = project.Id }, pr
        }
        catch (DomainException ex) { return BadRequest(new { error = ex.Message
    }

    // GET /api/projects/search?status=Active&page=1&pageSize=10
    [HttpGet("search")]
    public async Task<IActionResult> Search(
        [FromQuery] string? status, [FromQuery] int page = 1, [FromQuery] int pa
    {
        var result = await _projectService.GetPagedAsync(page, pageSize, status)
        return Ok(result);
    }
}
```

6.5 Configuración de EF Core: ApplicationDbContext.cs

Define el mapeo ORM, restricciones de base de datos e índices únicos mediante Fluent API.

```
// Índice único: OrdenId debe ser único dentro del mismo proyecto
entity.HasIndex(t => new { t.ProjectId, t.Order }).IsUnique();

// El estado se almacena como string ("Draft", "Active", "Completed")
entity.Property(p => p.Status).IsRequired().HasConversion<string>();

// Cascade delete: si se borra un proyecto, se borran sus tareas
entity.HasOne(t => t.Project)
      .WithMany(p => p.Tasks)
      .HasForeignKey(t => t.ProjectId)
      .OnDelete(DeleteBehavior.Cascade);
```

6.6 Configuración de JWT: Program.cs

```
// JWT Authentication configurado en el arranque de la aplicación
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options => {
        options.TokenValidationParameters = new TokenValidationParameters {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwtKey))
        };
    });
```

7. API — Endpoints

Método	Ruta	Descripción	Auth
POST	/api/auth/register	Registrar nuevo usuario	No
POST	/api/auth/login	Login → retorna JWT	No
GET	/api/projects/search	Listar proyectos paginados (filtro por estado)	✓ JWT
GET	/api/projects/summary	Resumen global de proyectos y tareas	✓ JWT
POST	/api/projects	Crear proyecto	✓ JWT
PUT	/api/projects/{id}	Actualizar proyecto	✓ JWT
DELETE	/api/projects/{id}	Eliminar proyecto	✓ JWT
PATCH	/api/projects/{id}/activate	Activar proyecto (requiere tareas)	✓ JWT
PATCH	/api/projects/{id}/complete	Completar proyecto	✓ JWT
GET	/api/projects/{id}/tasks	Listar tareas de un proyecto	✓ JWT
POST	/api/tasks/{projectId}	Crear tarea	✓ JWT
PUT	/api/tasks/{id}	Actualizar tarea	✓ JWT
DELETE	/api/tasks/{id}	Eliminar tarea	✓ JWT
PATCH	/api/tasks/{id}/complete	Marcar tarea como completada	✓ JWT

PATCH	/api/tasks/{id}/reorder	Cambiar orden de tarea	✓ JWT

8. Despliegue con Docker

El archivo `docker-compose.yml` orquesta tres servicios:

Servicio	Imagen / Build	Puerto	Descripción
`db`	postgres:16-alpine	5432	Base de datos PostgreSQL con healthcheck
`api`	Platform.Api/Dockerfile	5294	API REST — espera a que db esté healthy
`web`	Platform.Web/Dockerfile	5229	Interfaz MVC — depende de db y api

```yaml
services:
  db:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d management_platform"]
      interval: 5s
      retries: 5

  api:
    build: { context: ., dockerfile: Platform.Api/Dockerfile }
    ports: ["5294:8080"]
    depends_on:
      db: { condition: service_healthy }  # Espera DB antes de arrancar

  web:
    build: { context: ., dockerfile: Platform.Web/Dockerfile }
    ports: ["5229:8080"]
    depends_on: [db, api]
```

Comando de inicio:

```
docker compose up -d --build
```

Luego, aplicar migraciones:

```
dotnet ef database update --project Platform.Infrastructure --startup-project Platform.Api
```

9. Pruebas Unitarias

Las pruebas verifican las reglas de negocio de la capa Domain en total aislamiento (sin base de datos ni HTTP).

Test	Resultado Esperado
ActivateProject_WithTasks_ShouldSucceed	Estado cambia a Active
ActivateProject_WithoutTasks_ShouldFail	DomainException lanzada
CompleteProject_WithAllTasksCompleted_ShouldSucceed	Estado cambia a Completed
CompleteProject_WithPendingTasks_ShouldFail	DomainException lanzada
CreateTask_WithInvalidOrder_ShouldFail	DomainException lanzada (orden < 1)
DeleteProject_ShouldBeDeletedWhenEntityRemoved	Proyecto eliminado correctamente

```
// Ejemplo de test — regla de negocio: sin tareas no se puede activar
[Fact]
public void ActivateProject_WithoutTasks_ShouldFail()
{
    var project = Project.Create("Demo", "desc", Guid.NewGuid());

    var ex = Assert.Throws<DomainException>(() => project.Activate(taskCount: 0)

    Assert.Contains("al menos una tarea", ex.Message);
}

// Ejemplo de test — activación exitosa con tareas presentes
[Fact]
public void ActivateProject_WithTasks_ShouldSucceed()
{
    var project = Project.Create("Demo", "desc", Guid.NewGuid());

    project.Activate(taskCount: 1);

    Assert.Equal(ProjectStatus.Active, project.Status);
}
```